

Universidad Peruana
Cayetano Heredia

CARRERA DE INGENIERÍA INFORMÁTICA

Curso: SISTEMAS OPERATIVOS

Integrantes:

- Paloma Fernanda Velasquez (60921203)
- Daniel Fabricio Oliva Moscoso (73710164)
- Alex Jhosep Ccoyllo Sotelo (3476409)
- Jeral Alexander Cueva Zapata (74140816)
- Sofía Dayana Calva Becerra (60239714)
- Rosita Dayana Espinola Abanto (60825993)
- Marlon Steven Sarmiento Montalvo (75181595)

Grupo: 5
Profesor: Roberto Eduardo Burga Montañez
Semestre académico: 2026-2

Capítulo 1: Seguridad y Protección – Amenazas y Objetivos de Seguridad

1. **Actividad: Identificación de permisos de archivos**

   Objetivo: Reconocer riesgos de seguridad asociados a permisos incorrectos.

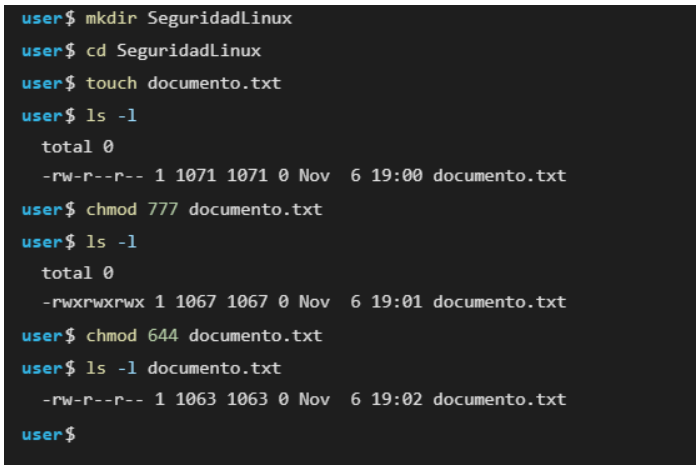
   Procedimiento

   mkdir SeguridadLinux
   cd SeguridadLinux

   touch documento.txt
   ls -l

   Modificar permisos:

   chmod 777 documento.txt
   ls -l



   Preguntas

   - **¿Qué riesgo implica otorgar permisos 777?**
     El permiso 777 permite que todos los usuarios lean, modifiquen y tengan permiso de ejecución sobre el archivo. Esto representa un riesgo porque personas no autorizadas podrían alterar o eliminar información.

   - **¿Qué usuarios pueden modificar el archivo?**
     Todos los usuarios: el propietario, los miembros del grupo y los demás usuarios, siempre que puedan acceder al archivo.

   - **¿Cuál sería una configuración más segura?**
     El permiso 644, porque solo el propietario puede modificar el archivo y los demás únicamente pueden leerlo.

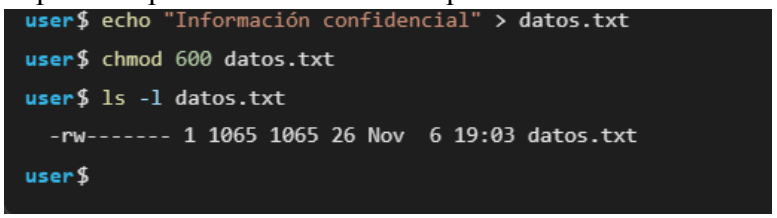
2. **Actividad: Protección de información**

   Crear un archivo confidencial:

   echo "Información confidencial" > datos.txt
   chmod 600 datos.txt
   ls -l datos.txt

   Evidencia

   Captura de pantalla mostrando los permisos.



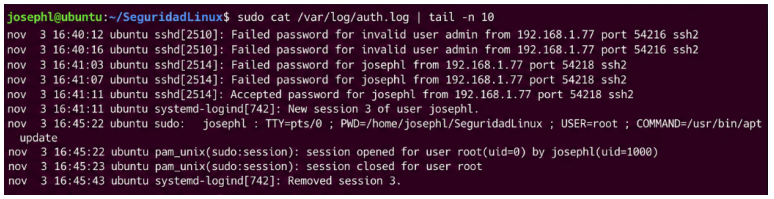
Capítulo 2: Intentos de Acceso No Autorizado

1. **Actividad: Monitoreo de intentos de acceso**

   Visualizar registros de autenticación:

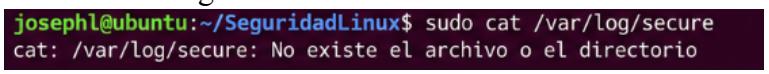
   Ubuntu/Debian:

   sudo cat /var/log/auth.log



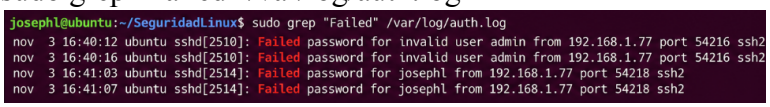
   CentOS/RHEL:

   sudo cat /var/log/secure



   Buscar intentos fallidos:

   sudo grep "Failed" /var/log/auth.log



   Preguntas

- **¿Cuántos intentos fallidos se registraron?**
  Se registraron 4 intentos fallidos de autenticación, identificados mediante el comando sudo grep "Failed" /var/log/auth.log. El resultado se confirmó utilizando sudo grep -c "Failed" /var/log/auth.log, que devolvió el valor 4.

- **¿Desde qué usuario se realizaron?**
  Los intentos fallidos estuvieron dirigidos a dos usuarios: admin y josephl. Se registraron dos intentos para cada uno, provenientes de la dirección IP 192.168.1.77.

2. **Actividad: Verificación de usuarios conectados**

   who

```
josephl@ubuntu:~/SeguridadLinux$ who
josephl      pts/0        2026-11-03 16:45   (192.168.1.50)
estudiante1  pts/1        2026-11-03 17:10   (192.168.1.77)
```

   w

```
josephl@ubuntu:~/SeguridadLinux$ w
 17:20:01 up  1:12,  2 users,  load average: 0,08, 0,11, 0,10
USER         TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
josephl      pts/0    192.168.1.50     16:45    2.00s  0.11s  0.03s bash
estudiante1  pts/1    192.168.1.77     17:10    0.00s  0.03s  0.01s bash
```

   Evidencia

   Registrar usuarios activos en el sistema.

   Evidencia de la Actividad 2: Se utilizaron los comandos who y w para identificar las sesiones activas del sistema. En la simulación se observaron dos usuarios conectados: josephl y estudiante1

Capítulo 3: Políticas y Mecanismos de Seguridad

1. **Actividad 1: Creación de usuarios**

   Crear un usuario:

   sudo useradd estudiante1
   sudo passwd estudiante1

```
daniel-oliva@daniel-oliva-VirtualBox:~$ sudo useradd estudiante1
[sudo: authenticate] Contraseña:
daniel-oliva@daniel-oliva-VirtualBox:~$ sudo passwd estudiante1
Nueva contraseña:
CONTRASEÑA INCORRECTA: No se ha proporcionado ninguna contraseña
Vuelva a escribir la nueva contraseña:
No se ha proporcionado ninguna contraseña.
passwd: Error de manipulación del testigo de autenticación
passwd: no se ha cambiado la contraseña
daniel-oliva@daniel-oliva-VirtualBox:~$ sudo passwd estudiante1
Nueva contraseña:
CONTRASEÑA INCORRECTA: De alguna manera, en la contraseña se lee el nombre del usuario
Vuelva a escribir la nueva contraseña:
passwd: contraseña actualizada correctamente
```

Verificar:

cat /etc/passwd | grep estudiante1

```
daniel-oliva@daniel-oliva-VirtualBox:~$ cat /etc/passwd | grep estudiante1
estudiante1:x:1002:1002::/home/estudiante1:/bin/sh
```

Preguntas

- **¿Por qué es importante asignar cuentas individuales?**
  La asignación de cuentas individuales es esencial porque constituye la base de la trazabilidad y auditoría, la responsabilidad personal, el principio de mínimo privilegio y el control de acceso granular; además, habilita el no repudio, mejora la detección de amenazas, facilita la gestión del ciclo de vida de identidades y permite cumplir con normativas como ISO 27001, NIST, GDPR o PCI-DSS. Sin cuentas individuales, las políticas de seguridad pierden eficacia, ya que no es posible atribuir acciones a personas concretas ni aplicar privilegios diferenciados, lo que favorece el uso indebido, las cuentas huérfanas y la falta de rendición de cuentas.

2. **Actividad: Configuración de políticas de contraseña**

   Visualizar políticas:

   sudo cat /etc/login.defs

```
daniel-oliva@daniel-oliva-VirtualBox:~$ sudo cat /etc/login.defs
[sudo: authenticate] Contraseña:
#
# /etc/login.defs - Configuration control definitions for the shadow package.
#

# REQUIRED for useradd/userdel/usermod
#   Directory where mailboxes reside, _or_ name of file, relative to the
#   home directory.  If you _do_ define MAIL_DIR and MAIL_FILE,
#   MAIL_DIR takes precedence.
#
#   Essentially:
#      - MAIL_DIR defines the location of users mail spool files
#        (for mbox use) by appending the username to MAIL_DIR as defined
#        below.
#      - MAIL_FILE defines the location of the users mail spool files as the
#        fully-qualified filename obtained by prepending the user home
#        directory before $MAIL_FILE
#
# NOTE: This is no more used for setting up users MAIL environment variable
#       which is, starting from shadow 4.0.12-1 in Debian, entirely the
#       job of the pam_mail PAM modules
#       See default PAM configuration files provided for
#       login, su, etc.
#
# This is a temporary situation: setting these variables will soon
# move to /etc/default/useradd and the variables will then be
# no more supported
MAIL_DIR        /var/mail
#MAIL_FILE      .mail

#
# Enable display of unknown usernames when login(1) failures are recorded.
#
# WARNING: Unknown usernames may become world readable.
# See #290803 and #298773 for details about how this could become a security
# concern
LOG_UNKFAIL_ENAB        no
```

Identificar:

PASS_MAX_DAYS
PASS_MIN_DAYS
PASS_WARN_AGE

```
# Password aging controls:
#
#       PASS_MAX_DAYS   Maximum number of days a password may be used.
#       PASS_MIN_DAYS   Minimum number of days allowed between password changes.
#       PASS_WARN_AGE   Number of days warning given before a password expires.
#
PASS_MAX_DAYS   99999
PASS_MIN_DAYS   0
PASS_WARN_AGE   7
```

Evidencia

Explicar la finalidad de cada parámetro.

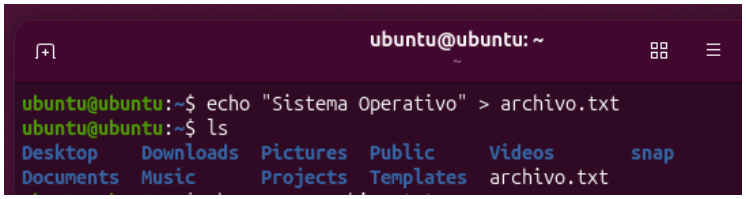
- PASS_MAX_DAYS: Número máximo de días que se puede utilizar una contraseña.
- PASS_MIN_DAYS: Número mínimo de días permitido entre cambios de contraseña.
- PASS_WARN_AGE: Número de días de antelación con los que se envía un aviso antes de que caduque una contraseña.

Capítulo 4: Validación

1. **Actividad: Verificación de integridad mediante hash**

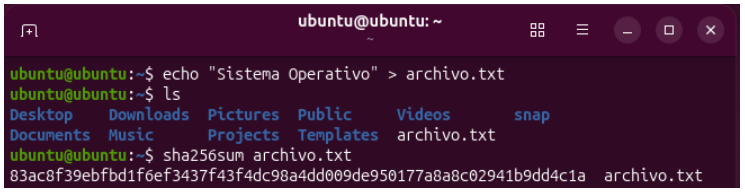
Crear archivo:

echo "Sistema Operativo" > archivo.txt



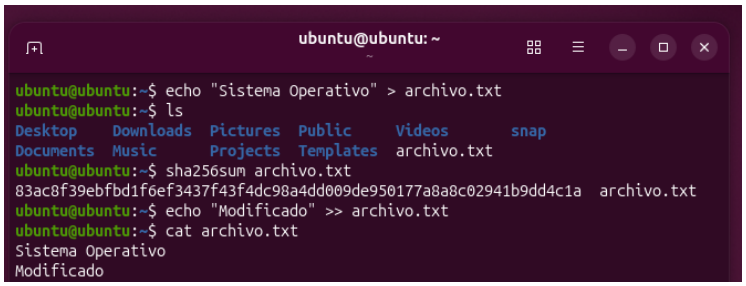
Generar hash:sha

sha256sum archivo.txt

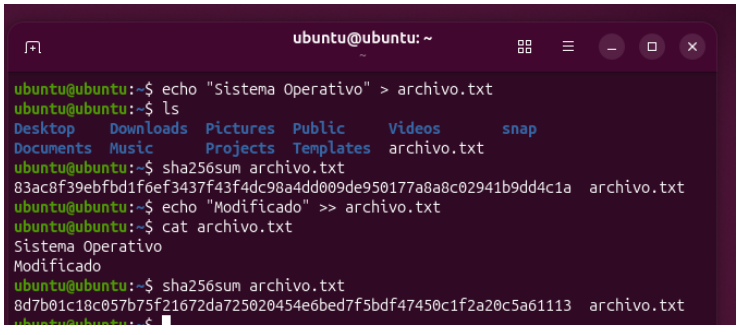


Modificar archivo:

echo "Modificado" >> archivo.txt



sha256sum archivo.txt



Preguntas

- **¿Cambió el hash?**
  Sí, el hash cambió por completo. Aunque solo se agregó una palabra, el nuevo código alfanumérico es totalmente diferente al primero.

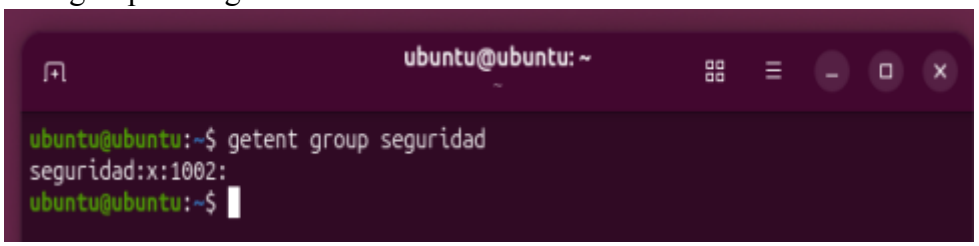
- **¿Qué demuestra este resultado?**
  Demuestra el principio de integridad. Cualquier alteración o modificación en el contenido de un archivo (por mínima que sea, incluso un solo espacio o letra) romperá el hash original. En seguridad informática, esto sirve para verificar que un archivo no ha sido manipulado ni alterado por terceros.

Capítulo 5: Protección y Control de Acceso

1. **Actividad: Gestión de grupos**

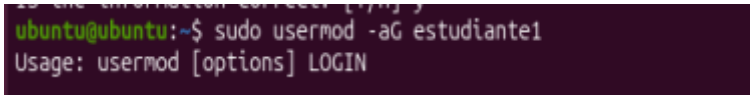
   Crear grupo:

   sudo groupadd seguridad



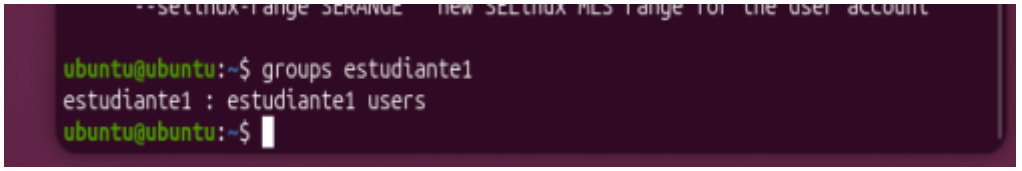
   Agregar usuario:

   sudo usermod -aG seguridad estudiante1



   Verificar:

   groups estudiante1



   Preguntas

   - ¿Qué ventajas ofrece el control por grupos?

   Permite administrar los permisos de varios usuarios de manera más fácil y segura. Facilita asignar permisos a todos los integrantes de un grupo, mejora la organización y evita accesos no autorizados a archivos y recursos del sistema.

   Actividad 2: Uso de ACL

   Instalar ACL:

   sudo apt install acl

```
ubuntu@ubuntu:~$ sudo apt install acl
acl is already the newest version (2.3.2-2).
acl set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
ubuntu@ubuntu:~$ getfacl --version
```

Crear archivo:

touch informe.txt

```
ubuntu@ubuntu:~$ touch informe.txt
ubuntu@ubuntu:~$ ls -l informe.txt
-rw-rw-r-- 1 ubuntu ubuntu 0 Nov  6 19:36 informe.txt
ubuntu@ubuntu:~$
```

Asignar permisos:

setfacl -m u:estudiante1:rwx informe.txt

Verificar:

getfacl informe.txt

Evidencia

Mostrar permisos ACL configurados.

```
ubuntu@ubuntu:~$ setfacl -m u:estudiante1:rwx informe.txt
ubuntu@ubuntu:~$ getfacl informe.txt
# file: informe.txt
# owner: ubuntu
# group: ubuntu
user::rw-
user:estudiante1:rwx
group::rw-
mask::rwx
other::r--
```

Capítulo 6: Modelos Formales de Protección

1. **Actividad: Aplicación del Principio de Mínimo Privilegio**

   Crear usuario:

   sudo useradd auditor

```
daniel-oliva@daniel-oliva-VirtualBox:~$ sudo useradd auditor
[sudo: authenticate] Contraseña:
daniel-oliva@daniel-oliva-VirtualBox:~$
```

   Asignar únicamente permisos de lectura:

chmod 444 datos.txt

```
daniel-oliva@daniel-oliva-VirtualBox:~$ chmod 444 datos.txt
chmod: no se puede acceder 'datos.txt': No existe el archivo o directorio
daniel-oliva@daniel-oliva-VirtualBox:~$ touch datos.txt
daniel-oliva@daniel-oliva-VirtualBox:~$ chmod 444 datos.txt
```

Probar acceso con:

su auditor

```
daniel-oliva@daniel-oliva-VirtualBox:~$ su auditor
Contraseña:
su: Fallo de autenticación
```

Intentar modificar:

echo "Prueba" >> datos.txt

```
daniel-oliva@daniel-oliva-VirtualBox:~$ echo "Prueba" >> datos.txt
bash: datos.txt: Permiso denegado
```

Preguntas

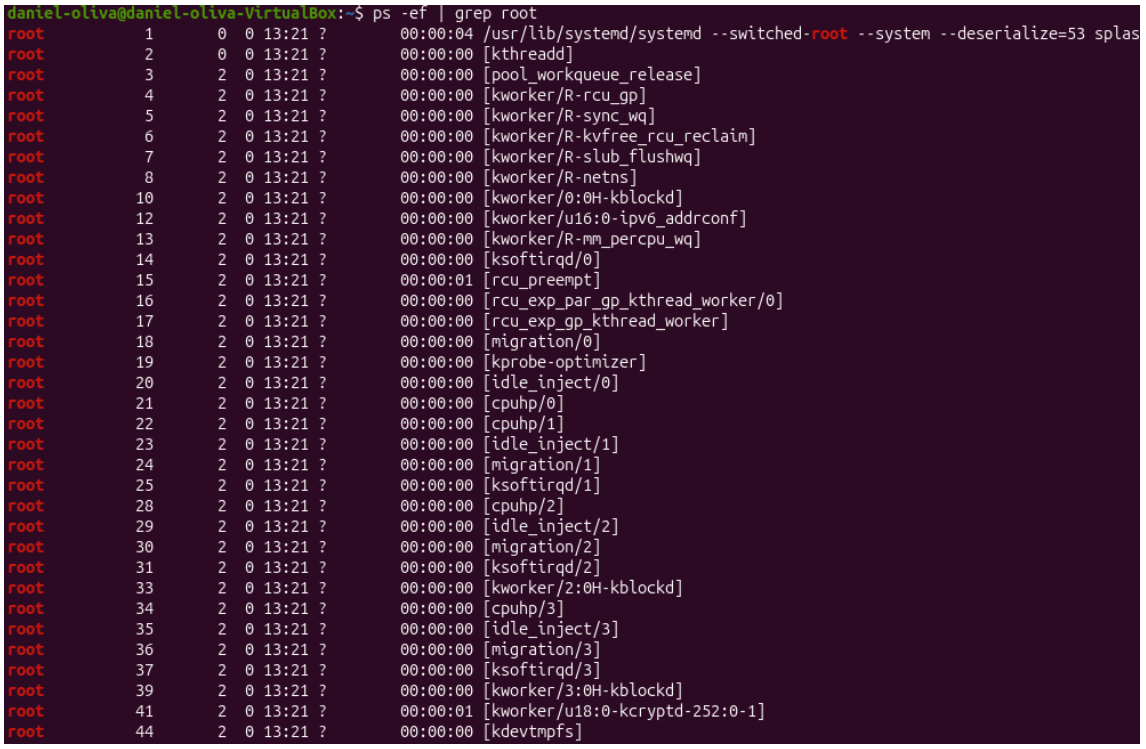
- **¿Se pudo modificar el archivo?**
  No. El comando chmod 444 datos.txt otorga permisos de solo lectura (lectura, lectura, lectura) para el dueño, el grupo y otros usuarios. Al iniciar sesión con el usuario auditor e intentar ejecutar echo "Prueba" >> datos.txt, el sistema operativo denegará la operación (arrojando un error de "Permiso denegado"), ya que el usuario no tiene el permiso de escritura (w) requerido para alterar el contenido del archivo.

- **¿Qué principio de seguridad se aplicó?**
  Se aplicó el Principio de Mínimo Privilegio (también conocido como menor privilegio). En los modelos formales de protección, este principio establece que a un sujeto (en este caso, el usuario auditor) solo se le deben conceder los permisos estrictamente necesarios para realizar su función (en este caso, únicamente leer el archivo), denegando explícitamente cualquier otro acceso no requerido (como la modificación o escritura). Esto minimiza la superficie de ataque y previene alteraciones no autorizadas en la integridad de la información.

2. **Actividad: Análisis de privilegios**

Identificar procesos ejecutados como root:

ps -ef | grep root



Evidencia

Seleccionar tres procesos y explicar su función.

- systemd: Se encarga de inicializar los componentes del sistema, gestionar los servicios (iniciar, detener, reiniciar demonios), montar sistemas de archivos y gestionar el arranque y apagado. Requiere privilegios de root porque necesita control total sobre el hardware y los recursos del sistema para poder orquestar el resto de procesos de manera segura.
- sshd (Secure Shell Daemon): Se queda a la escucha en el puerto 22 esperando conexiones entrantes, gestiona la autenticación de los usuarios y crea las sesiones cifradas. Requiere privilegios de root para poder escuchar en puertos privilegiados (menores a 1024) y para poder cambiar la identidad del proceso al usuario correspondiente una vez que este se autentica (aplicando el principio de mínimo privilegio).
- cron: Es el encargado de ejecutar tareas programadas de forma automática en segundo plano (por ejemplo, copias de seguridad, limpieza de logs, scripts de mantenimiento). Requiere privilegios de root porque debe ser capaz de leer los archivos de configuración de tareas (crontab) de cualquier usuario del sistema y ejecutar comandos en nombre de ellos (o del propio root) en los horarios establecidos, independientemente de quién haya iniciado sesión.

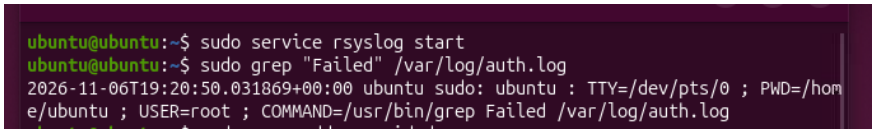
Actividad Integradora Final

Caso práctico

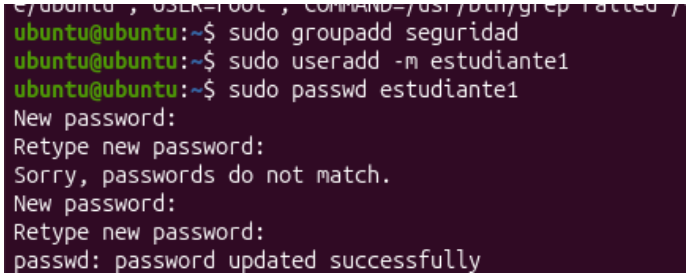
Una empresa detecta intentos de acceso no autorizado a un servidor Linux.

Desarrollo:

1. Se realiza el aseguramiento y auditoría de eventos del sistema. En primer lugar, se inicializa el servicio de registro centralizado mediante el comando sudo service rsyslog start. Acto seguido, se procede a inspeccionar el archivo de autenticación y seguridad de Linux usando sudo grep "Failed" /var/log/auth.log con el objetivo de rastrear de forma proactiva cualquier intento fallido de inicio de sesión o ejecución de comandos sospechosos en el sistema.



2. Se realiza la creación y configuración inicial del grupo y la cuenta de usuario en el sistema. Primero, se crea el grupo de control mediante sudo groupadd seguridad. Luego, se da de alta al nuevo usuario con el comando sudo useradd -m estudiante1, forzando la creación de su directorio personal. Finalmente, se establece su contraseña segura mediante sudo passwd estudiante1, logrando que las credenciales se actualicen correctamente tras validar la confirmación en el segundo intento (password updated successfully).



3. Se lleva a cabo la asignación del usuario al grupo de seguridad y el control estricto de accesos mediante Listas de Control de Acceso (ACL). Tras añadir a estudiante1 al grupo seguridad y verificarlo con groups, se genera un archivo de datos confidenciales protegido con chmod 600. Acto seguido, se define una regla específica mediante sudo setfacl -m u:estudiante1:rw datos_servidor.txt para otorgarle privilegios de lectura y escritura exclusivos a este usuario, comprobando la correcta implementación de la directiva y los permisos personalizados con getfacl.

```
passwd: password updated successfully
ubuntu@ubuntu:~$ sudo usermod -aG seguridad estudiante1
ubuntu@ubuntu:~$ groups estudiante1
estudiante1 : estudiante1 seguridad
ubuntu@ubuntu:~$ echo "Datos confidenciales de la empresa" > datos_servidor.txt
ubuntu@ubuntu:~$ chmod 600 datos_servidor.txt
ubuntu@ubuntu:~$ sudo setfacl -m u:estudiante1:rw datos_servidor.txt
ubuntu@ubuntu:~$ getfacl datos_servidor.txt
# file: datos_servidor.txt
# owner: ubuntu
# group: ubuntu
user::rw-
user:estudiante1:rw-
group::---
mask::rw-
other::---
```

4. Se ejecuta la auditoría de integridad de datos mediante el algoritmo criptográfico SHA-256. Primero, se genera el hash original del archivo para establecer su firma digital de referencia. Posteriormente, se simula una alteración o inyección maliciosa al introducir el texto "Intrusión" en el archivo. Al recalcular el hash con sha256sum, se demuestra un cambio radical en la cadena resultante, lo que comprueba matemáticamente que cualquier modificación en el contenido rompe la integridad del archivo y es fácilmente detectable.

```
ubuntu@ubuntu:~$ sha256sum datos_servidor.txt
9994612f1a5470884d4046a99396a24d6dd26b1609a91977f04aaf0baf951ec8  datos_servidor
.txt
ubuntu@ubuntu:~$ echo "Intrusiòn" >> datos_servidor.txt
ubuntu@ubuntu:~$ sha256sum datos_servidor.txt
2d35bfdcc92e012782152d0a657f8b7816a4b434fd34aa34b19f5927e7135416  datos_servidor
.txt
```

Conclusión:

A través del desarrollo de esta práctica, se validaron e implementaron los pilares fundamentales de la seguridad de la información en sistemas operativos Linux: confidencialidad, control de acceso e integridad de datos.

Mediante la correcta administración de usuarios y la implementación de Listas de Control de Acceso (ACL) con setfacl, se demostró cómo mitigar el riesgo de accesos no autorizados, garantizando el principio de mínimos privilegios sobre los recursos críticos del servidor. Asimismo, la integración del servicio de registros rsyslog y la inspección proactiva de archivos de autenticación evidenciaron la importancia de los mecanismos de auditoría para la detección oportuna de anomalías o fallos en el sistema.

Finalmente, la utilización del algoritmo criptográfico SHA-256 demostró ser una herramienta indispensable y matemáticamente infalible para el control de integridad. Se comprobó de manera práctica que cualquier alteración o inyección de contenido no autorizada, por mínima que sea, destruye por completo el valor del hash original. Esto subraya la necesidad crítica de implementar sistemas de monitoreo de integridad de archivos (FIM) para blindar y auditar la información sensible en entornos de producción reales.